意图路由

Claude Code 意图识别、意图路由与调度架构

目录

- 一、总体架构概览
- 二、输入分类与意图识别
  - 2.1 输入模式 (PromptInputMode)
  - 2.2 processUserInput 主入口
  - 2.3 关键词检测与 Ultraplan 路由
- 三、斜杠命令解析与路由
  - 3.1 斜杠命令解析器
  - 3.2 斜杠命令执行器 (processSlashCommand)
- 四、命令注册与 Skill 系统
  - 4.1 命令类型定义
  - 4.2 命令来源层级
  - 4.3 Skill 目录加载
  - 4.4 Skill 命令创建
  - 4.5 MCP Skill 加载
- 五、命令队列与优先级调度
  - 5.1 统一命令队列
  - 5.2 队列操作 API
  - 5.3 斜杠命令识别
- 六、查询引擎 (QueryEngine)
  - 6.1 架构概述
  - 6.2 submitMessage 核心流程
- 七、核心查询循环 (query.ts)
  - 7.1 主循环结构
  - 7.2 工具识别与流式执行
  - 7.3 模型回退机制
- 八、工具编排与路由 (Tool Orchestration)
  - 8.1 工具分类执行
  - 8.2 并发安全分区
  - 8.3 流式工具执行 (StreamingToolExecutor)
- 九、MCP (Model Context Protocol) 集成
  - 9.1 MCP 服务器连接
  - 9.2 MCP 工具路由
  - 9.3 MCP Skill
- 十、子代理系统 (AgentTool)
  - 10.1 子代理执行
  - 10.2 子代理隔离
  - 10.3 恢复机制
- 十一、Skill 搜索与推荐
  - 11.1 技能预取 (Skill Prefetch)
  - 11.2 技能发现流程
  - 11.3 技能索引缓存
- 十二、完整路由决策树
- 十三、关键设计模式
  - 13.1 懒加载 (Lazy Loading)
  - 13.2 记忆化缓存 (Memoization)
  - 13.3 事件信号 (Signal)
  - 13.4 条件编译 (Dead Code Elimination)
  - 13.5 统一队列 + React 订阅
- 十四、文件速查表
- 十五、总结

一、总体架构概览

Claude Code 的意图识别与路由系统是一个**多层级的管道式架构**，从用户输入到最终执行经历了以下阶段：

```
用户输入
    │
    ▼
┌──────────────────────────────────────────────────────┐
│  processUserInput (输入分类与预处理)                  │
│  src/utils/processUserInput/processUserInput.ts      │
└──────────────────────────────────────────────────────┘
    │
    ├─[ Ultraplan 关键词检测 ] ──────────────────────┐
    │  feature('ULTRAPLAN')                           │
    ├─[ Bash 模式分支 ] ──────────────────────────────┤
    │  mode === 'bash' → processBashCommand()         │
    ├─[ Slash 命令分支 ] ─────────────────────────────┤
    │  input.startsWith('/') → processSlashCommand()  │
    └─[ 普通 Prompt 分支 ] ───────────────────────────┤
       input → processTextPrompt()                    │
                  │
                  ▼
         ┌──────────────────┐
         │   Slash 命令解析与路由    │
         │   processSlashCommand    │
         └──────────────────┘
                  │
          ┌───────┴───────┐
          ▼       ▼       ▼
```

```
      [prompt]  [local]  [local-jsx]
         │         │          │
    ┌────┴────┐    │          │
    ▼         ▼    ▼          ▼
[inline]  [fork]  本地函数  React组件
 Skill     子代理  输出文本   渲染Ink
 文本展开   执行              UI
    └────┬────┘
         ▼
┌──────────────────────────────────────────┐
│  QueryEngine (查询引擎)                    │
│  src/QueryEngine.ts                       │
│  1. processUserInput() → 获取消息          │
│  2. fetchSystemPromptParts() → 构建系统提示  │
│  3. getSlashCommandToolSkills() → 加载技能列表 │
│  4. query() → 核心循环                     │
└──────────────────────────────────────────┘
                  │
                  ▼
┌──────────────────────────────────────────┐
│  query() 核心循环 (src/query.ts)           │
│  1. 上下文压缩 (snip/microcompact/autocompact)  │
│  2. callModel() → 流式 API 调用            │
│  3. 工具识别 (tool_use block)              │
│  4. runTools() → 工具执行                  │
│  5. 循环直到无需 follow-up                 │
└──────────────────────────────────────────┘
                  │
                  ▼
         ┌──────────────────┐
         │   工具编排与路由   │
         │ toolOrchestration.ts │
         └──────────────────┘
                  │
          ┌───────┴───────┐
          ▼       ▼       ▼
      [ 内置工具 ] [MCP 工具]   [AgentTool]
       Read/Edit  MCP Server   子代理执行
```

二、输入分类与意图识别

2.1 输入模式 (PromptInputMode)

源码位置: `src/types/textInputTypes.ts`

系统定义了三种核心输入模式，通过 `mode` 字段区分：

模式	含义	处理路径
`prompt`	普通用户提示词	`processTextPrompt()`
`bash`	Shell 命令（以 ! 开头）	`processBashCommand()`
`task-notification`	系统任务通知（后台任务完成等）	直接转换为 attachment 消息
`local-jsx`	内部命令	渲染为 Ink UI

2.2 processUserInput 主入口

源码位置: `src/utils/processUserInput/processUserInput.ts`

```
export async function processUserInput({
  input,           // string | ContentBlockParam[]
  mode,            // PromptInputMode
  setToolJSX,      // 设置工具 JSX 回调
  context,         // ProcessUserInputContext (ToolUseContext + LocalJSXCommandContext)
  querySource,     // 'repl_main_thread' | 'sdk' | 'compact' | ...
  skipSlashCommands, // 跳过斜杠命令解析 (用于远程桥接消息)
  ...
})
```

核心流程:

1. **图片预处理**: 对 ContentBlockParam[] 中的 image 块进行 resize/downsample
2. **Bridge 安全检查**: 如果是远程客户端(bridgeOrigin)发来的斜杠命令，验证 `isBridgeSafeCommand()`
3. **Ultraplan 关键词检测**: 检测 "plan" 等关键词，自动路由到 `/ultraplan`
4. **Bash 模式分支**: `mode === 'bash'` → 执行 shell 命令
5. **斜杠命令分支**: `input.startsWith('/')` → 解析并执行斜杠命令
6. **普通 Prompt 分支**: 所有其他输入 → 直接构建用户消息
7. **UserPromptSubmit Hooks**: 在返回前执行 hooks（如安全检查、上下文注入）

Note

分流策略：贪心优先还是语义优先?
`processUserInput` 采用的是**贪心字符串匹配优先**（Ultraplan 关键词、`startsWith('/')`、Bash 模式），而非语义级别的 LLM 分类：

- **优点**：零 LLM 调用开销、确定性 O(n) 时间复杂度（n = 输入长度）、无模型幻觉风险
- **缺点**：无法处理 "/plan with a twist" 这类歧义输入（会误触发 ultraplan）
- **设计哲学**：CLI 工具的用户输入本身就具有高度结构性（命令前缀、特殊符号），利用这个先验分布可以让简单规则达到很高的准确率——只有当字符串规则无法判断时（如用户输入 "plan" 而非 "/plan"），才通过 feature flag 暗启动态路由

这与 LLM Agent 中"一切皆 prompt"的路线形成鲜明对比。Claude Code 在这里选择了**局部贪心** + **全局语义**的混合架构：CLI 特定的入口点用规则，复杂语义判断交给 QueryEngine 中的 LLM。

2.3 关键词检测与 Ultraplan 路由

源码位置: `src/utils/ultraplan/keyword.ts`

```
// 检测用户输入中是否包含 ultraplan 关键词
export function hasUltraplanKeyword(input: string): boolean
// 将关键词替换为 "plan"
export function replaceUltraplanKeyword(input: string): string
```

Ultraplan 路由条件:

- `feature('ULTRAPLAN')` 必须启用
- `mode === 'prompt'`
- 非交互式会话跳过（headless/print 模式下不支持）
- 输入不以 `/` 开头
- 当前无 ultraplan session

三、斜杠命令解析与路由

3.1 斜杠命令解析器

源码位置: `src/utils/slashCommandParsing.ts`

```
export type ParsedSlashCommand = {
  commandName: string  // e.g. "commit", "mcp:tool (MCP)"
  args: string         // e.g. "-m 'fix: bug'"
  isMcp: boolean       // 是否为 MCP 命令
}

export function parseSlashCommand(input: string): ParsedSlashCommand | null
```

解析规则:

- 移除前缀 `/`
- 按空格分割，第一个词为命令名
- 如果第二个词是 `(MCP)`，则标记为 `isMcp: true`，命令名变为 `mcp:tool (MCP)`
- 剩余部分为参数

示例:

```
/commit -m 'fix bug'     → { commandName: 'commit', args: "-m 'fix bug'", isMcp: false }
/mcp:readFile (MCP) foo  → { commandName: 'mcp:readFile (MCP)', args: 'foo', isMcp: true }
```

3.2 斜杠命令执行器 (processSlashCommand)

源码位置: `src/utils/processUserInput/processSlashCommand.tsx` (约 78,000+ 字符，超大文件)

这是整个路由系统的核心模块，核心逻辑如下:

```
async function processSlashCommand(
  inputString,          // 完整斜杠命令字符串
  precedingInputBlocks, // 输入中的图片块
  imageContentBlocks,   // 处理后的图片
  attachmentMessages,   // 文件附件
  context,              // ProcessUserInputContext
  setToolJSX,
  uuid,
  isAlreadyProcessing,
  canUseTool,
): Promise<SlashCommandResult>
```

执行流程:

1. **解析命令**: `parseSlashCommand(inputString)` → `ParsedSlashCommand`
2. **命令查找**: `findCommand(commandName, context.options.commands)`
3. **命令可用性检查**: `meetsAvailabilityRequirement(cmd)` (检查是否需要 Claude.ai 订阅或 Console API)
4. **启用状态检查**: `isCommandEnabled(cmd)` (检查 feature flag、env 等)
5. **执行分支**:

分支 A: type = 'prompt' && context = 'inline' (Skill 文本展开)

```
// 位于 case 'prompt' 内部, if (command.context === 'fork') 之后
return await getMessagesForPromptSlashCommand(command, args, context, ...)
```

`getMessagesForPromptSlashCommand()` 实际工作:

- 调用 `cmd.getPromptForCommand()` 读取 SKILL.md 并做 `${1}` / `${name}` 参数替换
- 将展开后的文本内容包装成 `UserMessage` 加入消息历史
- 返回 `shouldQuery: true`，由 QueryEngine 走 LLM 处理

Skill 内容获取 (`createSkillCommand.getPromptForCommand`):

- 读取 `SKILL.md` 文件内容
- 执行参数替换 `${1}`, `${name}` → 用户输入值
- 替换 `${CLAUDE_SKILL_DIR}` 为技能目录
- 替换 `${CLAUDE_SESSION_ID}` 为当前会话 ID
- 执行内联 shell 命令 `!`command` (安全限制: MCP skill 不执行)
- 返回展开后的文本内容数组

分支 B: type = 'prompt' && context = 'fork' (子代理执行)

```
// 位于 case 'prompt' 内部, if (command.context === 'fork') 条件触发
return await executeForkedSlashCommand(command, args, context, ...)
```

子代理执行流程 (`executeForkedSlashCommand`):

- `prepareForkedCommandContext()`: 构建子代理上下文
- 如果 `feature('KAIROS')` 且 `kairosEnabled`: **异步后台执行**（fire-and-forget）
  - 等待 MCP 服务器就绪（最多 10s，轮询 200ms 间隔）
  - `runAgent()` 在后台运行子代理
  - 结果通过 `enqueuePendingNotification()` 以 `isMeta: true` 重新入队
  - 主线程立即返回，不阻塞用户
- 否则: **同步执行**（阻塞直到完成）

Note

Fire-and-Forget 子代理的公平性保证
异步 fork（KAIROS 模式）使用了 `enqueuePendingNotification` 将结果重新入队，这是一种**延迟调度**设计：

- 子代理在后台独立运行，**不阻塞**用户继续输入
- 结果以 `later` 优先级入队，保证用户后续输入（`next` 优先级）不会被饿死
- `isMeta: true` 标记使得结果在队列预览中可见但不自动弹出（`isQueuedCommandEditable` 返回 false）

时序不确定性：由于是 fire-and-forget，结果可能在用户输入多条消息后才到达。系统通过 `isMeta` 和 `isQueuedCommandVisible` 控制显示，通过 `popAllEditable` 将积压的可编辑命令合并到输入缓冲区，实现"看起来连续"的体验。这是**乐观并发**（Optimistic Concurrency）思想：不保证顺序，但保证进度+公平性。

与同步 fork 的对比：同步模式下父代理等待子代理完成（`for await...runAgent()`），结果直接合并到当前轮次。对用户来说感觉是"等待"，但不会有命令丢失或乱序的问题。KAIROS 模式用延迟换取响应性，是 feature flag 控制的渐进式能力升级。

分支 C: type === 'local' (本地命令执行)

```
case 'local': {
  const module = await cmd.load()  // 懒加载命令模块
  const result = await module.call(args, context)
  // result: { type: 'text' } | { type: 'compact' } | { type: 'skip' }
}
```

分支 D: type === 'local-jsx' (本地 UI 命令)

```
case 'local-jsx': {
  const module = await cmd.load()  // 懒加载 React 组件
  const node = await module.call(onDone, context, args)
  // 返回 React.ReactNode，渲染为 Ink UI
  // onDone 回调在命令完成时触发
}
```

四、命令注册与 Skill 系统

4.1 命令类型定义

源码位置: `src/types/command.ts`

```
// 命令基础属性
export type CommandBase = {
  name: string
  aliases?: string[]           // 命令别名
  description: string
  availability?: CommandAvailability[]  // ['claude-ai', 'console']
  isEnabled?: () => boolean    // 动态启用检查
  isHidden?: boolean           // 从 typeahead 隐藏
  whenToUse?: string           // 使用场景描述（用于 SkillTool 提示）
  version?: string
  disableModelInvocation?: boolean  // 禁止模型调用
  userInvocable?: boolean     // 用户能否通过 /skill-name 调用
  loadedFrom: LoadedFrom      // 来源: 'skills' | 'plugin' | 'bundled' | 'mcp' | ...
}

// 三种执行类型的命令
export type Command = CommandBase &
  (PromptCommand | LocalCommand | LocalJSXCommand)
```

PromptCommand (技能/斜杠命令):

```
export type PromptCommand = {
  type: 'prompt'
  progressMessage: string
  contentLength: number
  argNames?: string[]          // 形参名称
  allowedTools?: string[]     // 允许使用的工具白名单
  model?: string              // 指定执行模型
  source: SettingSource | 'builtin' | 'mcp' | 'plugin' | 'bundled'
  hooks?: HooksSettings       // 注册的 hook
  context?: 'inline' | 'fork' // 执行上下文
  agent?: string              // 子代理类型
  effort?: EffortValue        // 努力级别
  paths?: string[]            // 条件技能路径（glob 模式）
  getPromptForCommand(args, context): Promise<ContentBlockParam[]>
}
```

LocalCommand (本地命令):

```
type LocalCommand = {
  type: 'local'
  supportsNonInteractive: boolean
```

```
  load: () => Promise<LocalCommandModule>
}
```

LocalJSXCommand (本地 UI 命令):

```
type LocalJSXCommand = {
  type: 'local-jsx'
  load: () => Promise<LocalJSXCommandModule>
}
```

4.2 命令来源层级

源码位置: `src/commands.ts`

命令加载优先级（**后者覆盖前者**）:

```
1. bundled skills      → 内置技能（getBundledSkills）
2. builtin plugin skills → 内置插件技能（getBuiltinPluginSkillCommands）
3. skill dir commands  → .claude/skills/ 目录
4. workflow commands   → 工作流脚本（WORKFLOW_SCRIPTS feature）
5. plugin commands     → 插件提供的命令
6. plugin skills       → 插件提供的技能
7. COMMANDS() 内置命令  → 硬编码命令（如 /help, /clear）
```

加载函数:

```
// 主加载函数（带缓存）
export async function getCommands(cwd: string): Promise<Command[]>

// 缓存清理
export function clearCommandsCache(): void

// 按来源过滤的命令
export const getSkillToolCommands = memoize(async (cwd) => {
  // 返回模型可调用的 prompt 类型命令（用于 SkillTool）
})

export const getSlashCommandToolSkills = memoize(async (cwd) => {
  // 返回斜杠命令工具技能
})
```

4.3 Skill 目录加载

源码位置: `src/skills/loadSkillsDir.ts`

Skill 加载来源:

```
export const getSkillDirCommands = memoize(async (cwd) => {
  // 1. Managed skills: ~/.claude/ 策略目录
  // 2. User skills: ~/.claude/skills/
  // 3. Project skills: <project>/.claude/skills/
  // 4. Additional dirs: --add-dir 指定的目录
  // 5. Legacy commands: <project>/commands/ 目录（SKILL.md 格式）
})
```

条件技能 (Conditional Skills):

```
// 带有 paths frontmatter 的技能不会立即加载
// 只有当操作的文件匹配 glob 模式时才激活
export function activateConditionalSkillsForPaths(
  filePaths: string[],
  cwd: string,
): string[]

// 使用 ignore 库进行 gitignore 风格匹配
const skillIgnore = ignore().add(skill.paths)
if (skillIgnore.ignores(relativePath)) {
  dynamicSkills.set(name, skill)  // 激活技能
}
```

动态技能发现:

```
// 在文件操作过程中，发现嵌套的 .claude/skills/ 目录
export async function discoverSkillDirsForPaths(
  filePaths: string[],
  cwd: string,
): Promise<string[]>
// 向上遍历文件路径，找到匹配的 skills 目录

export async function addSkillDirectories(dirs: string[]): Promise<void>
// 加载并合并到动态技能 Map
```

4.4 Skill 命令创建

```
export function createSkillCommand({
  skillName,
  description,
  markdownContent,    // SKILL.md 内容
  allowedTools,       // 工具白名单
  argumentNames,      // 形参
  model,              // 指定模型
  context,            // 'inline' | 'fork'
```

```
  agent,              // 子代理类型
  effort,             // 努力级别
  paths,              // 条件路径
  shell,              // shell 配置
  ...
}): Command
```

Frontmatter 字段解析 (`parseSkillFrontmatterFields`):

```
---
name: display-name
description: 技能描述
when_to_use: 使用场景
allowed-tools: [Read, Edit, Bash]
arguments: [filename, content]
argument-hint: "<filename> <content>"
model: sonnet
effort: medium
context: fork        # inline(默认) | fork
agent: general-purpose
paths: ["src/**/*.ts"]  # 条件激活
user-invocable: true
disable-model-invocation: false
hooks:
  on_invocation:
    - ...
---
```

4.5 MCP Skill 加载

源码位置: `src/skills/mcpSkillBuilders.ts`

MCP 服务器可以提供 prompt 类型命令（技能），通过 `loadedFrom: 'mcp'` 标记。

安全限制: MCP skill 禁止执行内联 shell 命令（`!` 语法）。

五、命令队列与优先级调度

源码位置: `src/utils/messageQueueManager.ts`

5.1 统一命令队列

所有命令（用户输入、任务通知、孤立权限请求）都经过同一个队列:

```
const commandQueue: QueuedCommand[] = []
let snapshot: readonly QueuedCommand[] = Object.freeze([])
const queueChanged = createSignal()
```

优先级系统:

```
const PRIORITY_ORDER: Record<QueuePriority, number> = {
  now:   0,   // 立即执行（中断当前操作）
  next:  1,   // 下一轮执行（用户输入默认）
  later: 2,   // 最后执行（任务通知）
}
```

5.2 队列操作 API

```
// 入队
enqueue(command: QueuedCommand)           // 默认 'next' 优先级
enqueuePendingNotification(command)        // 默认 'later' 优先级

// 出队（按优先级 FIFO）
dequeue(filter?: (cmd) => boolean): QueuedCommand | undefined
dequeueAll(): QueuedCommand[]
dequeueAllMatching(predicate): QueuedCommand[]
remove(commandsToRemove: QueuedCommand[]): void
removeByFilter(predicate): QueuedCommand[]

// 查询
getCommandQueue(): QueuedCommand[]            // 复制
getCommandQueueSnapshot(): readonly QueuedCommand[]  // 冻结快照
getCommandsByMaxPriority(maxPriority): QueuedCommand[]
peek(filter?): QueuedCommand | undefined

// 清除
clearCommandQueue(): void  // 清空队列
resetCommandQueue(): void  // 重置（测试用）
```

5.3 斜杠命令识别

```
export function isSlashCommand(cmd: QueuedCommand): boolean {
  return (
    typeof cmd.value === 'string' &&
    cmd.value.trim().startsWith('/') &&
    !cmd.skipSlashCommands  // 远程消息跳过斜杠解析
  )
}
```

Remote Bridge 安全过滤:

```
export function isBridgeSafeCommand(cmd: Command): boolean {
  if (cmd.type === 'local-jsx') return false  // 禁止 Ink UI 命令
  if (cmd.type === 'prompt') return true       // skill 总是安全
  return BRIDGE_SAFE_COMMANDS.has(cmd)         // 本地命令白名单
}

export const BRIDGE_SAFE_COMMANDS: Set<Command> = new Set([
  compact, clear, cost, summary, releaseNotes, files
])
```

六、查询引擎 (QueryEngine)

源码位置: `src/QueryEngine.ts`

6.1 架构概述

`QueryEngine` 是会话生命周期管理者，每个会话一个实例:

```
export class QueryEngine {
  // ── 配置 ──────────────────────────────────────────────
  private config: QueryEngineConfig       // 构造时注入的完整配置快照

  // ── 会话状态（跨轮次持久）──────────────────────────────
  private mutableMessages: Message[]      // 会话消息历史（main loop 写入）
  private abortController: AbortController // 中断信号（ESC / abort）
  private permissionDenials: SDKPermissionDenial[]  // SDK 层的权限拒绝记录
  private totalUsage: NonNullableUsage   // 累计 API 使用量（token / cost）
  private hasHandledOrphanedPermission = false  // 是否已处理过孤立权限（仅一次）
  private readFileState: FileStateCache   // 文件读取状态缓存（变更检测用）

  // ── 发现与加载跟踪 ────────────────────────────────────
  // 轮次内 skill 发现跟踪（喂给 tengu_skill_tool_invocation 的 was_discovered）。
  // 每轮 submitMessage 开始时 clear，避免 SDK 长会话中无序增长。
  private discoveredSkillNames = new Set<string>()
  // 已加载的 .claude/skills 嵌套路径，防止重复加载
  private loadedNestedMemoryPaths = new Set<string>()
}
```

字段分组说明

- **配置**：`config` 是构造时注入的快照，所有运行时状态都存在其余字段中
- **会话状态**：消息历史、中断控制器、权限拒绝记录、API 使用量——这些跨 `submitMessage` 调用持久
- **发现与加载跟踪**：仅影响当轮决策（哪个 skill 可见、哪个目录已扫描），每轮重建防止状态泄漏

6.2 submitMessage 核心流程

```
async *submitMessage(
  prompt: string | ContentBlockParam[],
  options?: { uuid?: string; isMeta?: boolean },
): AsyncGenerator<SDKMessage, void, unknown>
```

流程:

1. **初始化**:
   - 清理 `discoveredSkillNames`（每轮重置）
   - 解析模型配置 (`userSpecifiedModel` 或 `getMainLoopModel()`)
   - 初始化 thinking 配置
2. **构建系统提示**:
   ```
   const { defaultSystemPrompt, userContext, systemContext }
     = await fetchSystemPromptParts({ tools, mainLoopModel, mcpClients, ... })
   ```
3. **处理用户输入**:
   ```
   const {
     messages: messagesFromUserInput,
     shouldQuery,    // 是否需要调用模型
     allowedTools,
     model: modelFromUserInput,
     resultText,     // 本地命令输出文本
   } = await processUserInput({ input: prompt, ... })
   ```
   - `shouldQuery: false` → 本地命令执行完毕，直接返回结果
   - `shouldQuery: true` → 需要模型处理
4. **更新工具权限上下文**:
   ```
   setAppState(prev => ({
     ...prev,
     toolPermissionContext: {
       ...prev.toolPermissionContext,
       alwaysAllowRules: { command: allowedTools }
     }
   }))
   ```
5. **加载 Skills 和 Plugins**（并行）:
   ```
   const [skills, { enabled: enabledPlugins }] = await Promise.all([
     getSlashCommandToolSkills(getCwd()),
     loadAllPluginsCacheOnly(),
   ```

```
])
```

6. **发送 SystemInit 消息**（包含所有可用工具/命令/技能/插件）：

```
yield buildSystemInitMessage({ tools, mcpClients, commands, agents, skills, plugins })
```

7. **进入查询循环**:

```
for await (const message of query({ messages, systemPrompt, ... })) {
  // 处理流式消息: assistant/user/progress/attachment/system
  // 更新 mutableMessages
  // yield 标准化消息给 SDK 消费者
}
```

七、核心查询循环 (query.ts)

源码位置: `src/query.ts`

7.1 主循环结构

```
async function* queryLoop(params: QueryParams): AsyncGenerator<...> {
  let state: State = {
    messages: params.messages,
    toolUseContext: params.toolUseContext,
    autoCompactTracking: undefined,
    turnCount: 1,
    ...
  }

  while (true) {
    // 1. 上下文压缩
    messagesForQuery = await snipModule?.snipCompactIfNeeded(messagesForQuery)
    messagesForQuery = await microcompact(messagesForQuery)
    messagesForQuery = await contextCollapse?.applyCollapsesIfNeeded(messagesForQuery)
    messagesForQuery = await autocompact(messagesForQuery)  // 自动压缩

    // 2. 调用模型
    for await (const message of callModel({ messages: messagesForQuery, ... })) {
      // 流式处理: 收集 assistantMessages, toolUseBlocks
      // 处理 withheld 消息（prompt-too-long, max-output-tokens）
    }

    // 3. 检查停止条件
    if (!needsFollowUp) {
      // 执行 stop hooks
      // 处理 token budget continuation
      return { reason: 'completed' }
    }

    // 4. 工具执行
    for await (const update of runTools(toolUseBlocks, ...)) {
      yield update.message
      if (update.newContext) updatedToolUseContext = update.newContext
    }

    // 5. 附件处理
    for await (const attachment of getAttachmentMessages(...)) {
      yield attachment
    }

    // 6. 循环
    state = { messages: [...messagesForQuery, ...assistantMessages, ...toolResults], ... }
  }
}
```

7.2 工具识别与流式执行

```
// 收集 tool_use blocks
const toolUseBlocks: ToolUseBlock[] = []
for (const content of message.message.content) {
  if (content.type === 'tool_use') {
    toolUseBlocks.push(content)
    needsFollowUp = true
  }
}

// 流式工具执行（可选功能）
if (streamingToolExecutor) {
  for (const toolBlock of msgToolUseBlocks) {
    streamingToolExecutor.addTool(toolBlock, message)
  }
}
```

7.3 模型回退机制

```
try {
  for await (const message of callModel({ model: currentModel, ... })) { ... }
} catch (innerError) {
  if (innerError instanceof FallbackTriggeredError && fallbackModel) {
    currentModel = fallbackModel
    attemptWithFallback = true
    // 清空 partial 消息，重新尝试
    continue
  }
  throw innerError
```

```
}
```

Note

查询循环：一个分层防御的上下文管理管道
queryLoop 的压缩阶段体现了**分层防御（Defense in Depth）** 的设计理念：

```
snipCompactIfNeeded → microcompact → contextCollapse → autocompact
```

每层只处理自己擅长的问题，职责单一：

阶段	策略	触发条件
`snipCompactIfNeeded`	摘要替换（删除细节，保留摘要）	`snip` 工具被调用
`microcompact`	逐轮增量压缩	消息数量超阈值
`contextCollapse`	软压缩（段落折叠，保留引用）	`feature('CONTEXT_COLLAPSE')`
`autocompact`	激进压缩（按 token 硬限制）	`feature('REACTIVE_COMPACT')` + token 超限

为什么不一次压缩到位？ 因为不同场景需要不同的压缩粒度：模型自己触发的 `snip` 是最精准的（模型知道什么信息冗余），而 `autocompact` 是最后的保底手段（按 token 预算硬切）。分层设计让每层可以独立调整，而不必在同一个压缩器里权衡多种策略。

八、工具编排与路由 (Tool Orchestration)

源码位置: `src/services/tools/toolOrchestration.ts`

8.1 工具分类执行

```
export async function* runTools(
  toolUseMessages: ToolUseBlock[],
  assistantMessages: AssistantMessage[],
  canUseTool: CanUseToolFn,
  toolUseContext: ToolUseContext,
): AsyncGenerator<MessageUpdate, void> {
  // 按并发安全性分组
  for (const { isConcurrencySafe, blocks } of partitionToolCalls(...)) {
    if (isConcurrencySafe) {
      // 并发执行只读操作（Read, Glob, Grep 等）
      yield* runToolsConcurrently(blocks, ...)
    } else {
      // 串行执行写操作（Edit, Bash, Write 等）
      yield* runToolsSerially(blocks, ...)
    }
  }
}
```

8.2 并发安全分区

内置工具定义了 `isConcurrencySafe` 属性:

- **只读工具**: `Read`, `Glob`, `Grep`, `WebSearch` 等 → 可并发
- **写操作工具**: `Edit`, `Write`, `Bash`, `NotebookEdit` 等 → 串行

8.3 流式工具执行 (StreamingToolExecutor)

可选功能 (`feature('STREAMING_TOOL_EXECUTION')`):

```
class StreamingToolExecutor {
  addTool(toolBlock: ToolUseBlock, assistantMessage: AssistantMessage): void
  getCompletedResults(): AsyncGenerator<ToolResult>
  getRemainingResults(): AsyncGenerator<ToolResult>  // 中断时生成合成结果
  discard(): void  // 丢弃未完成结果
}
```

优势: 用户在模型仍在流式输出时就看到工具开始执行结果，减少感知延迟。

Note

为什么不是全局最优分配？ 若允许打散重排（`[Bash, Read, Bash]` → `[Read], [Bash], [Bash]`），理论上可以增加并发度，但这需要 O(n!) 的排列枚举，当前策略是在**保序性**（用户期望工具按模型输出顺序执行）和**并发性**之间的合理折中。

`isConcurrencySafe` 的双重保护：`safeParse` + `try/catch` 两层防护——即使 schema 校验通过，`isConcurrencySafe()` 函数内部抛错（如 shell-quote 解析失败），保守地认为不并发安全。这是一种**失败即安全（Fail-Safe）**原则的应用：把不确定的操作降级为串行，而不是冒着数据竞争风险继续并发。

流式工具执行的背压控制：当模型快速输出一系列只读工具调用时，`StreamingToolExecutor` 需要在"尽快并发启动"和"不要让结果过时而需要重新入队"之间做权衡。源码中的 `getRemainingResults()` 在中断时生成合成结果，这是应对**部分失败**的韧性设计——部分工具完成、部分超时时，用户仍然能得到有意义的反馈。

九、MCP (Model Context Protocol) 集成

源码位置: `src/services/mcp/`

9.1 MCP 服务器连接

```
// 客户端类型
type MCPServerConnection = {
  name: string
  transport: Transport
  state: 'connecting' | 'connected' | 'error' | 'pending'
  tools: Tool[]           // 暴露给模型的工具
  resources: Resource[]
  prompts: MCPPrompt[]
```

```
}
```

9.2 MCP 工具路由

MCP 工具和其他内置工具一起被传给模型:

```
toolUseContext = {
  ...toolUseContext,
  options: {
    ...toolUseContext.options,
    tools: [...builtinTools, ...mcpTools]
  }
}
```

当模型调用 MCP 工具时:

1. `findToolByName(tools, block.name)` 找到工具
2. `runToolUse()` 执行工具
3. MCP 客户端通过 transport 发送请求到 MCP 服务器
4. 结果以 `tool_result` 消息返回

9.3 MCP Skill

当 MCP 服务器提供 `prompts` 类型资源时，可以被加载为 Skill:

```
if (cmd.loadedFrom === 'mcp' && cmd.type === 'prompt') {
  // getPromptForCommand 通过 MCP 协议获取 prompt 内容
  // 安全: MCP skills 禁止执行内联 shell 命令
}
```

十、子代理系统 (AgentTool)

源码位置: `src/tools/AgentTool/`

10.1 子代理执行

用于 `context: fork` 的 Skill 或显式的 `@agent-<type>` 提及:

```
// src/tools/AgentTool/runAgent.ts
for await (const message of runAgent({
  agentDefinition,   // { agentType, effort, ... }
  initialMessages,
  systemPrompt,
  parentContext,     // 从父代理继承的工具权限、文件状态等
  agentId,           // 唯一标识
  signal,            // AbortSignal
})) { ... }
```

10.2 子代理隔离

- **独立消息历史**: 每个子代理有自己独立的 `messages` 数组
- **工具权限继承**: 从父代理的 `toolPermissionContext` 继承，允许规则被传递
- **Token Budget**: 子代理有独立的 token 预算
- **Query Tracking**: 子代理的 `chainId` 与父代理关联，`depth` 递增

10.3 恢复机制

```
// src/tools/AgentTool/resumeAgent.ts
export async function resumeAgent(
  sessionId: string,
  agentId: string,
  ...
): Promise<void>
```

十一、Skill 搜索与推荐

源码位置: `src/services/skillSearch/`

11.1 技能预取 (Skill Prefetch)

当 `feature('EXPERIMENTAL_SKILL_SEARCH')` 启用时:

```
// 在模型流式输出和工具执行期间，异步发现相关技能
const pendingSkillPrefetch = skillPrefetch?.startSkillDiscoveryPrefetch(
  null, messages, toolUseContext
)
// 模型在下一轮 API 调用时看到新发现的技能
```

11.2 技能发现流程

1. **文件操作触发**: 当模型执行 Read/Write/Edit 操作时
2. **目录发现**: `discoverSkillDirsForPaths(filePaths, cwd)` 向上遍历
3. **条件激活**: `activateConditionalSkillsForPaths(filePaths, cwd)` 匹配 glob 模式
4. **动态加载**: `addSkillDirectories(newDirs)` 加载技能并通知

11.3 技能索引缓存

```
// 本地搜索索引（用于快速匹配）
export const getSkillIndex = memoize(async (cwd: string) => {
  const allCommands = await getCommands(cwd)
  return allCommands
    .filter(cmd => cmd.type === 'prompt' && cmd.hasUserSpecifiedDescription)
    .map(cmd => ({
      name: cmd.name,
      description: cmd.description,
      whenToUse: cmd.whenToUse,
      // 用于向量搜索或关键词匹配
    }))
})
```

十二、完整路由决策树

```
用户输入 "input"
    │
    ▼
┌─ input.startsWith('/')? ─┐
│  是                       │  否
▼                           ▼
parseSlashCommand(input)  ┌─ Ultraplan 关键词? ──────────────┐
    │                     │  是 → /ultraplan 命令              │
    ▼                     └─ 否 → mode === 'bash'? ──────────┤
findCommand(name)        │    是 → processBashCommand()      │
    │                     └─ 否 → processTextPrompt()        │
    ▼                                  │
命令存在? ─┐                           │
  是      │ 否                         ▼
  ▼       ▼              shouldQuery = true
meetsAvailability  → 返回 "Unknown skill"
    │                 错误消息
    ▼
isCommandEnabled?
    │
    ├─ type === 'prompt'
    │    │
    │    ├─ context === 'fork' → executeForkedSlashCommand()
    │    │                         ├─ KAIROS? → 异步后台
    │    │                         └─ 其他 → 同步执行
    │    │
    │    └─ context === 'inline' → cmd.getPromptForCommand()
    │                                        │
    │                                        ▼
    │                              shouldQuery = true
    │                              消息加入 mutableMessages
    │
    ├─ type === 'local'
    │    → cmd.load() → module.call()
    │          └─ { type: 'text' } → resultText
    │          └─ { type: 'compact' } → compactionResult
    │          └─ { type: 'skip' } → shouldQuery = false
    │
    └─ type === 'local-jsx'
         → cmd.load() → module.call(onDone, context, args)
               └─ React.ReactNode → 渲染 Ink UI
                    onDone() 回调触发后续流程
```

十三、关键设计模式

13.1 懒加载 (Lazy Loading)

- `local` 和 `local-jsx` 命令通过 `load()` 函数懒加载，避免启动时加载所有依赖
- `insights.ts` (113KB, 3200 行) 使用懒加载: `await import('./commands/insights.js')`
- 大部分功能模块使用 `require()` 条件加载，配合 `feature()` 实现 Tree-shaking

13.2 记忆化缓存 (Memoization)

```
const loadAllCommands = memoize(async (cwd) => { ... })
const getSkillToolCommands = memoize(async (cwd) => { ... })
const getSlashCommandToolSkills = memoize(async (cwd) => { ... })
```

Note

记忆化缓存的代价：新鲜度 vs. 性能的权衡
`memoize` 的默认行为是**永不失效**（除非手动清除），这在 CLI 会话场景下完全合理——用户在一个会话中不会新增/删除 skill 目录，重复调用应返回缓存结果。

潜在的缓存雪崩（Cache Avalanche）：若多个 `memoized` 函数共享底层状态（如 `conditionalSkills`、`dynamicSkills`），缓存清除的时机至关重要。当前通过 `onDynamicSkillsLoaded` 回调链式清除：`dynamicSkills` 变化 → Signal 触发 → 多个 memoized 缓存同步失效，保证了一致性。

异步 memoize 的陷阱：`memoize(async fn)` 包装异步函数时，**首次调用返回 Promise**（而不是等待完成）。若在 Promise resolved 之前有第二个调用，lodash 的默认 memoize 会返回**同一个 Promise 实例**（引用相等），这是正确的行为——防止并发请求重复执行。但如果 Promise rejected 后再调用，lodash memoize **不会重试**（返回缓存的 rejected Promise），这在网络瞬时故障时可能导致永久性失败。Claude Code 的 `getSkillDirCommands` 是幂等的（读文件系统），所以这个陷阱不会触发。

更精细的策略：对于非幂等操作，可以考虑带 TTL 的 memoize（`memoizeTimeout`）或基于事件的缓存失效（`evict-on-signal`），当前实现通过显式 `clearSkillCaches()` 和 `onDynamicSkillsLoaded` 回调链达到了类似效果。

13.3 事件信号 (Signal)

```
const skillsLoaded = createSignal()
// 动态技能加载完成后触发
skillsLoaded.emit()
// 其他模块订阅
```

```
onDynamicSkillsLoaded(() => { clearCommandMemoizationCaches() })
```

13.4 条件编译 (Dead Code Elimination)

```
const proactive = feature('PROACTIVE') || feature('KAIROS')
  ? require('./commands/proactive.js').default
  : null
// 外部构建版本中，未启用的功能代码被完全消除
```

13.5 统一队列 + React 订阅

命令队列是模块级单例，通过 `createSignal()` 实现事件通知，React 组件通过 `useSyncExternalStore(subscribeToCommandQueue, getCommandQueueSnapshot)` 订阅状态变更。

十四、文件速查表

功能	文件路径
意图识别主入口	`src/utils/processUserInput/processUserInput.ts`
斜杠命令执行器	`src/utils/processUserInput/processSlashCommand.tsx`
斜杠命令解析	`src/utils/slashCommandParsing.ts`
普通 Prompt 处理	`src/utils/processUserInput/processTextPrompt.ts`
命令定义/加载	`src/commands.ts`
命令类型定义	`src/types/command.ts`
Skill 加载	`src/skills/loadSkillsDir.ts`
Skill 创建	`src/skills/loadSkillsDir.ts` (createSkillCommand)
队列管理	`src/utils/messageQueueManager.ts`
查询引擎	`src/QueryEngine.ts`
核心查询循环	`src/query.ts`
工具编排	`src/services/tools/toolOrchestration.ts`
工具执行	`src/services/tools/toolExecution.ts`
MCP 客户端	`src/services/mcp/client.ts`
MCP 类型定义	`src/services/mcp/types.ts`
AgentTool 主入口	`src/tools/AgentTool/AgentTool.tsx`
子代理运行	`src/tools/AgentTool/runAgent.ts`
子代理恢复	`src/tools/AgentTool/resumeAgent.ts`
内置 Agent 定义	`src/tools/AgentTool/builtInAgents.ts`
技能搜索	`src/services/skillSearch/`
MCP Skill 构建器	`src/skills/mcpSkillBuilders.ts`
Skill 搜索本地	`src/services/skillSearch/localSearch.ts`
Skill 预取	`src/services/skillSearch/prefetch.ts`
系统提示构建	`src/utils/queryContext.ts`
流式工具执行器	`src/services/tools/StreamingToolExecutor.ts`
权限管理	`src/hooks/useCanUseTool.ts`
权限判断	`src/utils/permissions/permissions.ts`

十五、总结

Claude Code 的意图识别与路由架构是一个**高度模块化、分层递进**的设计:

1. **输入层** (`processUserInput`): 通过模式识别（Ultraplan 关键词、Slash 命令、Bash 模式）将用户输入分流到不同处理路径
2. **命令解析层**: 统一的命令定义模型支持 `prompt`、`local`、`local-jsx` 三种执行类型，通过 `findCommand()` 和 `meetsAvailabilityRequirement()` 实现灵活的路由
3. **技能层**: 基于 frontmatter 的声明式技能系统，支持条件激活、参数替换、内联 shell 执行、子代理 fork 等高级特性
4. **调度层**: 基于优先级的统一命令队列 (`now`/`next`/`later`) 确保用户输入不被系统通知饿死
5. **执行层**: `QueryEngine` 管理会话生命周期，`query.ts` 实现流式 API 调用与工具执行的循环，`toolOrchestration.ts` 处理并发安全分区
6. **扩展层**: MCP 协议集成将外部工具无缝纳入工具集，Plugin 系统支持第三方扩展

整个架构的设计亮点包括: 懒加载减少启动开销、记忆化缓存避免重复加载、条件编译实现 Tree-shaking、Signal 模式实现松耦合事件通知、以及异步后台执行实现非阻塞的用户体验。

Note

架构设计的元主题：渐进式复杂性（Progressive Complexity）
纵观整个架构，最核心的设计哲学不是任何一个具体模式，而是一种**延迟复杂性的纪律**：

- **分流用规则**：O(1) 字符串检查够用，就不用 LLM
- **分区用贪心**：线性扫描够用，就不用最优分配
- **缓存用 memoize**：重复加载确实发生，再加缓存
- **并发用条件**：并发确实带来性能收益，再拆并发安全/非安全
- **异步用 feature flag**：后台执行稳定了，再推广到所有用户

每一个"为什么不用 X"的答案，都是对**工程实用性**的坚持：不过度设计，不过早优化，根据实际规模选择最简单的方案。这与 LLM Agent 领域常见的"先用最强模型解决一切"的思路形成对照——Claude Code 选择在合适的层级使用最合适的工具，让规则引擎、文件系统约定、LLM 各司其职。